

EC200U&EG91xU Series QuecOpen(SDK) **HTTP(S) Development Guide**

LTE Standard Module Series

Version: 1.1

Date: 2023-11-23

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-02-23	Herry GENG	Creation of the document
1.0	2021-12-07	Herry GENG	First official release
1.1	2023-11-23	Kruskal ZHU	- Updated the document name based on the unified QuecOpen naming. - Added applicable modules EG912U-GL and EG915U series. - Added new members to the enumeration http_option_e (Chapter 3.3.2.1). - Added ql_httpc_continue_dload() function (Chapter 3.3.3). - Add new members HTTP_INFO_DATE and HTTP_INFO_PARTITION_LEN to the enumeration http_info_e (Chapter 3.3.6.1). - Add a note indicating that ql_httpc_release() does not support some callback functions (Chapter 3.3.7).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules	6
2 Calling Process	7
3 HTTP(S) API	9
3.1. Header File	9
3.2. API Overview	9
3.3. API Description	10
3.3.1. ql_httpc_new	10
3.3.1.1. http_client_event_cb_t	10
3.3.1.2. http_event_id_e	11
3.3.1.3. http_error_code_e	11
3.3.2. ql_httpc_setopt	12
3.3.2.1. http_option_e	13
3.3.2.2. http_method_e	16
3.3.2.3. https_verify_level_e	17
3.3.2.4. http_client_write_data_cb_t	17
3.3.2.5. http_client_read_data_cb_t	18
3.3.3. ql_httpc_continue_dload	19
3.3.4. ql_httpc_formadd	19
3.3.4.1. http_formtopt_e	20
3.3.5. ql_httpc_perform	20
3.3.6. ql_httpc_getinfo	21
3.3.6.1. http_info_e	21
3.3.7. ql_httpc_release	22
4 Example	24
5 Appendix References	25

Table Index

Table 1: Applicable Modules..... 6

Table 2: API Overview 9

Table 3: Related Documents 25

Table 4: Terms and Abbreviations 25

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It outlines HTTP API, related calling process and includes example for Quectel EC200U series and EG91xU family modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 Calling Process

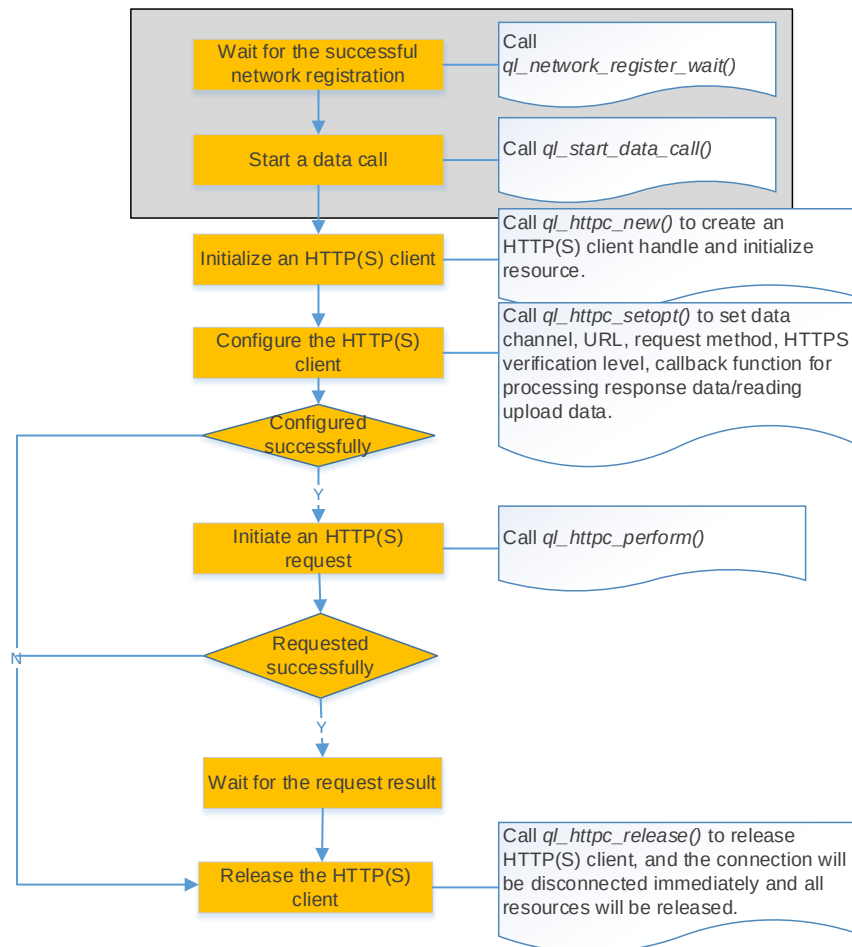


Figure 1: HTTP(S) API Calling Process

HTTP(S) service receives and sends data based on the specified data channel. Therefore, before sending the HTTP(S) request, you must first wait for the successful network registration and perform data calling to establish a data channel (as shown in the gray part in the above figure). If the data channel has been established in other tasks, you can skip the network registration and calling steps and call *ql_httpc_new()* directly to create an HTTP(S) client handle, then call *ql_httpc_setopt()* to configure the parameters such as established data channel (the default data channel is 1), URL, HTTP(S) request method, HTTP(S) verification level and the customized HTTP(S) header.

When you call `ql_httpc_perform()` to send an HTTP(S) request, the function returns the results immediately, and the corresponding callback function will be called when the relevant events occur. Thus, the next HTTP(S) operation can only be executed after the HTTP(S) is disconnected.

NOTE

1. EC200U series and EG91xU family modules only support HTTP/1.1 and also support GET, POST, PUT and HEAD requests.
2. An HTTP(S) client can only execute one HTTP(S) request at a time. The next request can be executed only after the current request is handled. To execute several HTTP(S) requests at the same time, you can create several HTTP(S) clients.
3. For more details about the `ql_network_register_wait()` and `ql_start_data_call()` data call functions, see ***document [2]***.

3 HTTP(S) API

3.1. Header File

ql_http_client.h, the header file of HTTP(S) API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_httpc_new()</i>	Creates a new HTTP(S) client handle and initializes the HTTP(S) client resources.
<i>ql_httpc_setopt()</i>	Configures HTTP(S) client attributes.
<i>ql_httpc_continue_dload()</i>	Restores the HTTP(S) download response corresponding to the HTTP(S) client handle.
<i>ql_httpc_formadd()</i>	Configures the attributes of HTTP(S) form. It is valid only when <i>HTTP_CLIENT_OPT_METHOD</i> is set to <i>HTTP_METHOD_POST</i> .
<i>ql_httpc_perform()</i>	Sends an HTTP(S) request.
<i>ql_httpc_getinfo()</i>	Gets the key value specified in the HTTP(S) header.
<i>ql_httpc_release()</i>	Releases HTTP(S) client resources.

3.3. API Description

3.3.1. ql_httpc_new

This function creates a new HTTP(S) client handle and initializes the HTTP(S) client resources.

- **Prototype**

```
int ql_httpc_new(http_client_t *client, http_client_event_cb_t cb, void *arg)
```

- **Parameter**

client:

[Out] HTTP(S) client handle.

cb:

[In] Callback function for HTTP(S) request event. See **Chapter 3.3.1.1** for details.

arg:

[In] Callback parameter of the HTTP(S) request event.

- **Return Value**

See **Chapter 3.3.1.3** for result codes.

3.3.1.1. http_client_event_cb_t

This function is the callback function for HTTP(S) request event. It is called when the related events occur after an HTTP(S) request is sent.

- **Prototype**

```
typedef void(*http_client_event_cb_t)(http_client_t *client, int evt, int evt_code, void *arg)
```

- **Parameter**

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

evt:

[In] HTTP(S) event type. See **Chapter 3.3.1.2** for details.

evt_code:

[In] Result code of HTTP(S) event. See **Chapter 3.3.1.3** for details.

arg:

[In] Callback parameter of the HTTP(S) request event. It is set by *ql_httpc_new()*.

● Return Value

None

3.3.1.2. http_event_id_e

The enumeration of HTTP(S) event types:

```
typedef enum{
    HTTP_EVENT_SESSION_ESTABLISH        = 0,
    HTTP_EVENT_RESPONSE_STATE_LINE      = 1,
    HTTP_EVENT_SESSION_DISCONNECT       = 2,
}http_event_id_e
```

● Member

Member	Description
<i>HTTP_EVENT_SESSION_ESTABLISH</i>	Establish an HTTP(S) session connection.
<i>HTTP_EVENT_RESPONSE_STATE_LINE</i>	Receive HTTP(S) response header. After receiving the response header, response code, content length and other parameter values can be obtained through <i>ql_httpc_getinfo()</i> . See Chapter 3.3.6 for details.
<i>HTTP_EVENT_SESSION_DISCONNECT</i>	Disconnect an HTTP(S) session.

3.3.1.3. http_error_code_e

The enumeration of HTTP(S) event result codes:

```
typedef enum{
    HTTP_SUCCESS                = 0,
    HTTP_ERR_UNKOWN             = -1,
    HTTP_ERR_WOUNDBLOCK         = -2,
    HTTP_ERR_INVALID_PARAM      = -3,
    HTTP_ERR_OUT_OF_MEM         = -4,
    HTTP_ERR_BUSY               = -5,
    HTTP_ERR_BAD_REQUEST        = -6,
    HTTP_ERR_SOCKE_CONNECT_FAIL = -7,
```

```

    HTTP_ERR_DNS_FAIL           = -8,
    HTTP_ERR_SOCKET_EXCEPTION   = -9,
    HTTP_ERR_TIMEOUT            = -10,
}http_error_code_e

```

● Member

Member	Description
<i>HTTP_SUCCESS</i>	Successful execution.
<i>HTTP_ERR_UNKOWN</i>	Unknown error.
<i>HTTP_ERR_WOUNDBLOCK</i>	Current operation is not completed.
<i>HTTP_ERR_INVALID_PARAM</i>	Invalid parameter.
<i>HTTP_ERR_OUT_OF_MEM</i>	Out of memory.
<i>HTTP_ERR_BUSY</i>	System is busy.
<i>HTTP_ERR_BAD_REQUEST</i>	Wrong request.
<i>HTTP_ERR_SOCKE_CONNECT_FAIL</i>	Failed to connect socket.
<i>HTTP_ERR_DNS_FAIL</i>	Failed to parse DNS.
<i>HTTP_ERR_SOCKET_EXCEPTION</i>	Socket is abnormal.
<i>HTTP_ERR_TIMEOUT</i>	Request times out.

3.3.2. ql_httpc_setopt

This function configures the HTTP(S) client attributes.

● Prototype

```
int ql_httpc_setopt(http_client_t *client, int opt_tag, ...)
```

● Parameter

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

opt_tag:

[In] Attribute tag. See **Chapter 3.3.2.1** for details.

...

[In] Attribute value. See **Chapter 3.3.2.1** for details.

- **Return Value**

See **Chapter 3.3.1.3** for result codes.

3.3.2.1. http_option_e

The enumeration of HTTP(S) attribute tags:

```
typedef enum{
    HTTP_CLIENT_OPT_PDPCID = 1,
    HTTP_CLIENT_OPT_SSLCTXID,
    HTTP_CLIENT_OPT_BASIC_AUTH,
    HTTP_CLIENT_OPT_REQUEST_HEADER,
    HTTP_CLIENT_OPT_WRITE_HEADER,
    HTTP_CLIENT_OPT_INTERVAL_TIME,
    HTTP_CLIENT_OPT_METHOD,
    HTTP_CLIENT_OPT_WRITE_FUNC,
    HTTP_CLIENT_OPT_WRITE_DATA,
    HTTP_CLIENT_OPT_READ_FUNC,
    HTTP_CLIENT_OPT_READ_DATA,
    HTTP_CLIENT_OPT_UPLOAD_LEN,
    HTTP_CLIENT_OPT_URL,
    HTTP_CLIENT_OPT_SSL_VERIFY_LEVEL,
    HTTP_CLIENT_OPT_SSL_CACERT_PATH,
    HTTP_CLIENT_OPT_SSL_OWN_CERT_PATH,
    HTTP_CLIENT_OPT_URI,
    HTTP_CLIENT_OPT_SIM_ID,
    HTTP_CLIENT_OPT_SSL_SNI,
    HTTP_CLIENT_OPT_SSL_VERSION,
    HTTP_CLIENT_OPT_SSL_HS_TIMEOUT,
    HTTP_CLIENT_OPT_SSL_IGNORE_LOCALTM,
    HTTP_CLIENT_OPT_SSL_IGNORE_INVALID_CERT_SIGN,
    HTTP_CLIENT_OPT_SSL_IGNORE_CERT_ITEM,
    HTTP_CLIENT_OPT_SSL_IGNORE_MULTI_CERTCHAIN_VERIFY,
    HTTP_CLIENT_OPT_RAW_REQUEST,
    HTTP_CLIENT_OPT_RAW_FILE
}http_option_e
```

- Member

Member	Description
<i>HTTP_CLIENT_OPT_PDPCID</i>	Set the data channel used by HTTP(S) client, namely the PDP context ID used for executing data call. See document [2] for details. Parameter type: int.
<i>HTTP_CLIENT_OPT_SSLCTXID</i>	Set the SSL context ID used by HTTPS client. Parameter type: int. Range: 0–5.
<i>HTTP_CLIENT_OPT_BASIC_AUTH</i>	Set the username and password for HTTP(S) basic authentication. Format: Username:Password. Maximum length: 255 bytes. Parameter type: char *.
<i>HTTP_CLIENT_OPT_REQUEST_HEADER</i>	Set the customized HTTP(S) request header. This member can be configured repeatedly to set several customized HTTP(S) requests. Parameter type: char*.
<i>HTTP_CLIENT_OPT_WRITE_HEADER</i>	Set whether to save HTTP(S) response header or not. 0 No 1 Yes Parameter type: int.
<i>HTTP_CLIENT_OPT_INTERVAL_TIME</i>	Set the maximum interval for awaiting HTTP(S) response data. Range: 1–65536. Unit: second Parameter type: int.
<i>HTTP_CLIENT_OPT_METHOD</i>	Set HTTP(S) request method. Parameter type: <i>http_method_e</i> . (See Chapter 3.3.2.2 for details.)
<i>HTTP_CLIENT_OPT_WRITE_FUNC</i>	Set the callback function for processing HTTP(S) response data. Parameter type: <i>http_client_write_data_cb_t</i> . (See Chapter 3.3.2.4 for details)
<i>HTTP_CLIENT_OPT_WRITE_DATA</i>	Set the parameter of the callback function that processes the HTTP(S) response data. Parameter type: void *.
<i>HTTP_CLIENT_OPT_READ_FUNC</i>	Set the callback function for reading the data uploaded by HTTP(S). It is valid only when <i>HTTP_CLIENT_OPT_METHOD</i> is set to <i>HTTP_METHOD_POST/HTTP_METHOD_PUT</i> . Parameter type: <i>http_client_read_data_cb_t</i> . (See Chapter 3.3.2.5 for details.)

<i>HTTP_CLIENT_OPT_READ_DATA</i>	Set the parameter of callback function that reads the data uploaded by HTTP(S). Parameter type: void *.
<i>HTTP_CLIENT_OPT_UPLOAD_LEN</i>	Set the length of the data uploaded by HTTP(S). It is valid only when <i>HTTP_CLIENT_OPT_METHOD</i> is set to <i>HTTP_METHOD_POST/HTTP_METHOD_PUT</i> . Parameter type: int.
<i>HTTP_CLIENT_OPT_URL</i>	Set the URL to be accessed by HTTP(S) client. URL should start with <i>https://</i> or <i>http://</i> . Parameter type: char *.
<i>HTTP_CLIENT_OPT_SSL_VERIFY_LEVEL</i>	Set HTTPS verification level. Parameter type: int. See Chapter 3.3.2.3 for details.
<i>HTTP_CLIENT_OPT_SSL_CACERT_PATH</i>	Set the path of HTTPS CA certificate. Parameter type: char *.
<i>HTTP_CLIENT_OPT_SSL_OWCERT_PATH</i>	Set the path of HTTPS local certificate. Parameter type: char *.
<i>HTTP_CLIENT_OPT_URI</i>	Set the URI to be accessed by HTTP(S) client. Parameter type: char*.
<i>HTTP_CLIENT_OPT_SIM_ID</i>	Set the path of HTTPS local certificate. That is, the (U)SIM card used for executing a data call. Parameter type: int.
<i>HTTP_CLIENT_OPT_SSL_SNI</i>	Configure whether HTTPS enables the Server Name Indication (SNI) feature. Parameter type: int. 0 Disable 1 Enable
<i>HTTP_CLIENT_OPT_SSL_VERSION</i>	Set the SSL version information of HTTPS. See document [3] for details.
<i>HTTP_CLIENT_OPT_SSL_HS_TIMEOUT</i>	Set the timeout of the HTTPS handshake. Parameter type: int. Range: 10–300 s.
<i>HTTP_CLIENT_OPT_SSL_IGNORE_LOCAL_TM</i>	Set whether HTTPS ignores the local time. Parameter type: int. 0 Do not ignore 1 Ignore
<i>HTTP_CLIENT_OPT_SSL_IGNORE_INVALID_CERT_SIGN</i>	Set whether HTTPS ignores the check of SSL certificate sent by the server. Parameter type: int 0 Do not ignore 1 Ignore
<i>HTTP_CLIENT_OPT_SSL_IGNORE_CERT_ITEM</i>	Set whether HTTPS ignores the check of SSL certificate items sent by the server. Parameter type: int. 0 Do not ignore

	1 Ignore
<i>HTTP_CLIENT_OPT_SSL_IGNORE_MULTI_CERTCHAIN_VERIFY</i>	Set whether HTTPS ignores SSL multi-level certificate chain verification. Parameter type: int. 0 Do not ignore 1 Ignore
<i>HTTP_CLIENT_OPT_RAW_REQUEST</i>	Enable or disable custom HTTP(S) request header information. Parameter type: int. 0 Disable 1 Enable
<i>HTTP_CLIENT_OPT_RAW_FILE</i>	Set the local file name for POST requests to the HTTP(S) server. Parameter type: char*.

3.3.2.2. http_method_e

The enumeration of HTTP(S) request methods:

```
typedef enum {
    HTTP_METHOD_NONE,
    HTTP_METHOD_GET,
    HTTP_METHOD_POST,
    HTTP_METHOD_PUT,
    HTTP_METHOD_HEAD,
    HTTP_METHOD_LAST
} http_method_e
```

● Member

Member	Description
<i>HTTP_METHOD_NONE</i>	No request.
<i>HTTP_METHOD_GET</i>	HTTP GET request. Request the specified page information and return the entity.
<i>HTTP_METHOD_POST</i>	HTTP POST request. Submit data from the client to the server for processing requests (such as submitting a form or uploading a file).
<i>HTTP_METHOD_PUT</i>	HTTP PUT request. The data transmitted from the client to the server replaces the specified document content.
<i>HTTP_METHOD_HEAD</i>	HTTP HEAD request. Request only the page header.

<code>HTTP_METHOD_LAST</code>	Last request.
-------------------------------	---------------

3.3.2.3. `https_verify_level_e`

The enumeration of HTTPS verification levels:

```
typedef enum{
    HTTPS_VERIFY_NONE          = 0,
    HTTPS_VERIFY_SERVER        = 1,
    /*HTTPS_VERIFY_SERVER*/
    HTTPS_VERIFY_SERVER_CLIENT = 2,
}https_verify_level_e
```

● Member

Member	Description
<code>HTTPS_VERIFY_NONE</code>	No verification
<code>HTTPS_VERIFY_SERVER</code>	Verify the server
<code>HTTPS_VERIFY_SERVER_CLIENT</code>	Two-way verification between client and server. The local certificate needs to be uploaded when this level is set. In addition, if the client is not verified by the server, this member equals <code>HTTPS_VERIFY_SERVER</code> .

3.3.2.4. `http_client_write_data_cb_t`

This callback function is called automatically to handle data after receiving HTTP(S) response data.

● Prototype

```
typedef int(*http_client_write_data_cb_t)(http_client_t *client, void *arg, char *data, int size, unsigned char end)
```

● Parameter

client:

[In] HTTP(S) client handle, which is obtained through `ql_httpc_new()`.

arg:

[In] Callback parameters set by attribute tag `HTTP_CLIENT_OPT_WRITE_DATA` of `ql_httpc_setopt()`. See **Chapter 3.3.2.1** for details.

data:

[In] Data to be processed.

size:

[In] Length of the data to be processed. Unit: byte.

end:

[In] End flag of the data packet.

0 The current packet is not the last one.

1 The current packet is the last one.

● Return Value

Length of the processed data.

3.3.2.5. http_client_read_data_cb_t

This callback function is called automatically to read the data to be uploaded after receiving an HTTP(S) POST or PUT request.

● Prototype

```
typedef int(*http_client_read_data_cb_t)(http_client_t *client, void *arg, char *data, int size)
```

● Parameter

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

arg:

[In] Callback parameters set by attribute tag *HTTP_CLIENT_OPT_READ_DATA* of *ql_httpc_setopt()*. See **Chapter 3.3.2** for details.

data:

[In] Buffer for storing data to be uploaded.

size:

[In] The size of buffer for storing the data to be uploaded. Unit: byte.

● Return Value

Length of the read data.

3.3.3. ql_httpc_continue_dload

This function restores the HTTP(S) download response corresponding to the HTTP(S) client handle.

- **Prototype**

```
int ql_httpc_continue_dload(http_client_t *client)
```

- **Parameter**

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

- **Return Value**

See **Chapter 3.3.1.3** for result codes.

3.3.4. ql_httpc_formadd

This function configures the attribute of HTTP(S) form. It is valid only when *HTTP_CLIENT_OPT_METHOD* is set to *HTTP_METHOD_POST*. See **Chapter 3.3.2.2** for details.

- **Prototype**

```
int ql_httpc_formadd(http_client_t *client, int opt_tag, ...)
```

- **Parameter**

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

opt_tag:

[In] Attribute tag. See **Chapter 3.3.4.1** for details.

...:

[In] Attribute value. See **Chapter 3.3.4.1** for details.

- **Return Value**

See **Chapter 3.3.1.3** for result codes.

3.3.4.1. http_formtopt_e

The enumeration of HTTP(S) form header attributes:

```
typedef enum{
    HTTP_FORM_NAME = 1,
    HTTP_FORM_FILENAME = 2,
    HTTP_FORM_CONTENT_TYPE = 3,
}http_formtopt_e
```

● Member

Member	Description
<i>HTTP_FORM_NAME</i>	Set the storage format of the data uploaded on server to file or image, etc. Parameter type: char*.
<i>HTTP_FORM_FILENAME</i>	Set the name of the uploaded data saved on the server. It is valid only when <i>HTTP_FORM_NAME</i> is set to file. Parameter type: char*.
<i>HTTP_FORM_CONTENT_TYPE</i>	Set the content type corresponding to the uploaded data. It is valid only when <i>HTTP_FORM_NAME</i> is set to file. Parameter type: char*.

3.3.5. ql_httpc_perform

This function sends an HTTP(S) request.

● Prototype

```
int ql_httpc_perform(http_client_t *client)
```

● Parameter

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

● Return Value

See **Chapter 3.3.1.3** for result codes.

NOTE

If the return value is not *HTTP_SUCCESS*, it indicates function execution failure. If yes, you can check whether the request result is successful through *http_client_event_cb_t()* since this is an asynchronous function.

3.3.6. ql_httpc_getinfo

This function gets the key value specified in the HTTP(S) header.

- **Prototype**

```
int ql_httpc_getinfo(http_client_t *client, int info,...)
```

- **Parameter**

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

info:

[In] Attribute tag. See **Chapter 3.3.6.1** for details.

...:

[Out] Attribute value. See **Chapter 3.3.6.1** for details.

- **Return Value**

See **Chapter 3.3.1.3** for result codes.

3.3.6.1. http_info_e

The enumeration of HTTP(S) response header attributes:

```
typedef enum{
/*Response code*/
    HTTP_INFO_RESPONSE_CODE = 0,
    HTTP_INFO_LOCATION,
    HTTP_INFO_DATE,
    HTTP_INFO_CONTENT_LEN,
    HTTP_INFO_CHUNK_ENCODE,
    HTTP_INFO_ACCEPT_RANGES,
    HTTP_INFO_PARTION_LEN,
}http_info_e
```

- Member

Member	Description
<i>HTTP_INFO_RESPONSE_CODE</i>	HTTP(S) response code. Parameter type: int*.
<i>HTTP_INFO_LOCATION</i>	Get the redirected URL. This value can be obtained only when the response code is 3xx. Parameter type: char*.
<i>HTTP_INFO_DATE</i>	Get date information. This value can be obtained only when the HTTP(S) response header contains date content. Parameter type: char*.
<i>HTTP_INFO_CONTENT_LEN</i>	Get content length. This value can be obtained only when HTTP(S) response header contains the content length. Parameter type: int*.
<i>HTTP_INFO_CHUNK_ENCODE</i>	Get whether the response body is in chunk encoding format or not. Parameter type: int *. 0 No 1 Yes
<i>HTTP_INFO_ACCEPT_RANGES</i>	Get whether the sever receives Range request or not. Parameter type: int *. 0 No 1 Yes
<i>HTTP_INFO_PARTION_LEN</i>	Get the full length of the requested data from the HTTP response field Content-Range. Parameter type: int *.

3.3.7. ql_httpc_release

This function releases HTTP(S) client resources.

- Prototype

```
int ql_httpc_release(http_client_t *client)
```

- Parameter

client:

[In] HTTP(S) client handle, which is obtained through *ql_httpc_new()*.

- Return Value

See **Chapter 3.3.1.3** for result codes.

NOTE

Since calling `ql_httpc_release()` in the callback function releases the current HTTP(S) client resources, subsequent use of the released resources by other HTTP(S) functions may result in module abnormalities. Therefore, `ql_httpc_release()` does not support `http_client_event_cb_t()`, `http_client_write_data_cb_t()`, and `http_client_read_data_cb_t()` callback functions. For more information about releasing HTTP(S) client resources by calling `ql_httpc_release()`, see **Chapter 4**.

4 Example

For details about HTTP(S) example, see the files *http_demo.c*, *http_post_demo.c*, *https_get_demo.c* and *https_post_demo.c* in the *components\ql-application\http* directory in the application layer of CSDK.

5 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_Data_Call_API_Reference_Manual
[3] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_SSL_Development_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CA	Certificate Authority
DNS	Domain Name Server
HTTP(S)	Hypertext Transfer Protocol
IoT	Internet of Things
RTOS	Real-Time Operating System
SDK	Software Development Kit
URI	Uniform Resource Identifier
URL	Uniform Resource Locator